COMPLETE JAVASCRIPT NOTES

Beginner to Intermediate • Simple explanations • Examples • Outputs • Interview points

How to use these notes: Read one topic, type the examples yourself, predict the output before running the code, and then practise the questions at the end.

Topics included

- JavaScript basics, variables, data types and operators
- Conditions, loops, functions, arrays and objects
- Strings, scope, hoisting, closures and the this keyword
- DOM, events, browser storage and error handling
- Modern JavaScript (ES6+), promises and async/await
- Common interview questions and practice exercises

1. Introduction to JavaScript

Definition: JavaScript is a programming language used to make websites interactive and dynamic. It can also run outside the browser using environments such as Node.js.

Technology	Main purpose
HTML	Creates the structure/content of a webpage.
CSS	Controls the appearance and layout.
JavaScript	Adds behaviour and interactivity.

Examples of what JavaScript can do

- Respond when a user clicks a button.
- Validate a login or signup form.
- Show, hide or update webpage content.
- Fetch information from a server.
- Build web servers and command-line tools with Node.js.

Adding JavaScript to HTML

Inline: JavaScript is written in an HTML attribute. Useful for tiny demonstrations, but usually avoid it in larger projects.

```
<button onclick="alert('Hello!')">Click me</button>
```

Internal: JavaScript is written inside a script element.

```
<script>
  console.log("Hello, JavaScript!");
</script>
```

External: JavaScript is stored in a separate .js file. This is usually the cleanest approach.

```
<!-- index.html -->
<script src="script.js" defer></script>

// script.js
console.log("Hello from an external file");
```

Tip: The defer attribute downloads the script while HTML is being parsed and runs it after the document has been parsed.

2. Variables: var, let and const

Definition: A variable is a named place used by a program to refer to a value.

```
var city = "Mumbai";
let age = 21;
const country = "India";
```

Feature	var	let	const
Can be redeclared in the same scope?	Yes	No	No
Can be reassigned?	Yes	Yes	No
Scope	Function/global	Block	Block

Feature	var	let	const
Must have a value at declaration?	No	No	Yes

Examples

```
let score = 10;
score = 20;          // allowed

const pi = 3.14;
// pi = 4;           // TypeError: reassignment is not allowed

const user = { name: "Asha" };
user.name = "Riya";  // allowed: object property is changed
```

Use **const** by default. Use **let** when you need to reassign. Avoid **var** in modern code unless you have a specific reason.

Important: const prevents assigning a new value to the variable. It does not make an object or array deeply immutable.

3. Data Types

A data type describes the kind of value being used. JavaScript has primitive types and object types.

Primitive types

Type	Meaning	Example
Number	Integers and decimal numbers	let price = 99.5;
String	Text	let name = "Asha";
Boolean	true or false	let isReady = true;
Undefined	A declared variable with no assigned value	let result;
Null	An intentional absence of a value	let selected = null;
BigInt	Very large integers	let big = 123n;
Symbol	A unique identifier	let id = Symbol("id");

Object types

Objects store properties, arrays store ordered collections, and functions are callable objects.

```
const student = { name: "Asha", age: 21 };
const marks = [80, 90, 95];

function greet() {
  console.log("Hello");
}
```

typeof operator

typeof returns a string describing the type of a value.

```
console.log(typeof 42);          // "number"
console.log(typeof "hi");         // "string"
console.log(typeof true);         // "boolean"
console.log(typeof undefined);    // "undefined"
console.log(typeof {});           // "object"
console.log(typeof null);         // "object" (historic JavaScript quirk)
console.log(typeof function(){}); // "function"
```

Remember: null is a primitive value even though typeof null returns "object". This is a long-standing historical behaviour.

4. Type Conversion and Coercion

Type conversion means deliberately changing a value from one type to another. **Type coercion** is conversion JavaScript performs automatically in some operations.

```
// Explicit conversion
Number("25");       // 25
String(25);         // "25"
Boolean(1);         // true
parseInt("42px");   // 42
parseFloat("3.5");  // 3.5

// Automatic coercion
console.log("5" + 2); // "52" (string concatenation)
console.log("5" - 2); // 3 (numeric conversion)
```

Truthy and falsy values

In a condition, JavaScript treats values as truthy or falsy. The main falsy values are: false, 0, -0, 0n, empty string (""), null, undefined and NaN. Almost everything else is truthy, including empty arrays and empty objects.

```
if ([]) {
  console.log("This runs: an empty array is truthy");
}
```

5. Operators

Category	Operators / purpose
Arithmetic	+ - * / % ** ++ --
Assignment	= += -= *= /= %=
Comparison	== === != !== > < >= <=
Logical	&& (AND), || (OR), ! (NOT)
Nullish coalescing	?? uses the right side only for null or undefined
Ternary	condition ? valueIfTrue : valueIfFalse
Type	typeof, instanceof

Equality: == versus ===

```
console.log(10 == "10");  // true: values are compared after coercion
console.log(10 === "10"); // false: types are different
```

Prefer **===** and **!==** because they avoid unexpected type conversion.

Logical operators

```
const age = 22;
const hasID = true;

console.log(age >= 18 && hasID); // true
console.log(age < 18 || hasID);  // true
console.log(!hasID);             // false
```

Ternary operator

```
const age = 17;
const status = age >= 18 ? "Adult" : "Minor";
console.log(status); // "Minor"
```

Nullish coalescing and optional chaining

```
const input = null;
const name = input ?? "Guest"; // "Guest"

const user = {};
console.log(user.address?.city); // undefined, no error
```

6. Conditional Statements

Conditions let a program choose which code to execute.

```
const marks = 82;

if (marks >= 90) {
  console.log("Grade A");
} else if (marks >= 75) {
  console.log("Grade B");
} else {
  console.log("Keep practising");
}
```

switch

Use switch when one expression is compared with several possible values.

```
const day = 2;

switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  default:
    console.log("Unknown day");
}
```

Why break? It prevents execution from continuing into the next case. Without break, switch can fall through.

7. Loops

Loops repeat a block of code while a condition or iteration rule allows it.

for loop

```
for (let i = 1; i <= 3; i++) {
  console.log(i);
}
// 1, 2, 3
```

while loop

```
let i = 1;
while (i <= 3) {
  console.log(i);
  i++;
}
```

do...while loop

A do...while loop runs its body at least once because it checks the condition afterwards.

```
let count = 5;
do {
  console.log(count);
} while (count < 3);
// prints 5 once
```

for...of and for...in

```
const colors = ["red", "blue"];
for (const color of colors) {
  console.log(color); // values
}

const person = { name: "Asha", age: 21 };
for (const key in person) {
  console.log(key, person[key]); // property keys and values
}
```

Loop	Best used for
for	Repeating with a counter or known iteration pattern.
while	Repeating while a condition remains true.
do...while	When the body must run at least once.
for...of	Reading values from an iterable such as an array or string.
for...in	Enumerating an object's property keys.

8. Functions

Definition: A function is a reusable block of code that performs a task.

Function declaration

```
function greet() {
  console.log("Hello!");
}
greet();
```

Parameters, arguments and return

```
function add(a, b) { // a and b are parameters
  return a + b;
}
const total = add(10, 20); // 10 and 20 are arguments
console.log(total); // 30
```

console.log() displays a value. **return** sends a value back to the caller and ends that function call.

Function expression

```
const sayHi = function() {
  console.log("Hi!");
};
sayHi();
```

Arrow function

```
const multiply = (a, b) => a * b;
console.log(multiply(4, 5)); // 20
```

An arrow function with one parameter can omit parentheses: `x => x * 2`. Arrow functions do not have their own this or arguments binding.

Callback and higher-order function

A **callback** is a function passed to another function. A **higher-order function** accepts a function as an argument or returns a function.

```
function processNumber(value, callback) {
  return callback(value);
}

const doubled = processNumber(5, n => n * 2);
console.log(doubled); // 10
```

IIFE

An Immediately Invoked Function Expression runs as soon as it is created.

```
(function() {
  console.log("Runs immediately");
})();
```

Default parameters and rest parameters

```
function welcome(name = "Guest") {
  console.log("Hello " + name);
}
welcome(); // Hello Guest

function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3)); // 6
```

9. Arrays

An array stores an ordered list of values. Array indexes start at 0.

```
const fruits = ["Apple", "Mango", "Banana"];
console.log(fruits[0]); // Apple
console.log(fruits.length); // 3
```

Method	What it does	Example
push(x)	Adds to the end	arr.push(40)
pop()	Removes from the end	arr.pop()
unshift(x)	Adds to the beginning	arr.unshift(5)

Method	What it does	Example
shift()	Removes from the beginning	arr.shift()
includes(x)	Checks whether a value exists	arr.includes(20)
indexOf(x)	Returns first matching index, or -1	arr.indexOf(20)
slice(a,b)	Returns a shallow portion; end excluded	arr.slice(1,3)
splice(a,n)	Changes original array by removing n items at a	arr.splice(1,2)
join(sep)	Joins elements into a string	arr.join('', '')
concat(other)	Returns a combined array	arr.concat(other)

slice versus splice

```
const values = [10, 20, 30, 40];

const part = values.slice(1, 3);
console.log(part);   // [20, 30]
console.log(values); // original unchanged

values.splice(1, 2);
console.log(values); // [10, 40] (original changed)
```

map, filter, forEach and reduce

map() returns a new array containing the result of transforming each element.

```
const nums = [1, 2, 3];
const doubled = nums.map(num => num * 2);
console.log(doubled); // [2, 4, 6]
```

Here, **num** is simply the callback parameter. During each call it represents the current array element.

filter() returns a new array containing only elements that pass a test.

```
const nums = [1, 2, 3, 4, 5];
const evens = nums.filter(num => num % 2 === 0);
console.log(evens); // [2, 4]
```

forEach() runs a callback for each element. It returns undefined, so it is not used to build a transformed array.

```
["a", "b"].forEach(item => console.log(item));
```

reduce() combines array values into one result, such as a sum.

```
const nums = [1, 2, 3, 4];
const total = nums.reduce((sum, num) => sum + num, 0);
console.log(total); // 10
```

Method	Returns	Common purpose
map()	New array	Transform every element.
filter()	New array	Keep matching elements.
forEach()	undefined	Perform an action for each element.
reduce()	One accumulated result	Sum, count, group or combine values.

sort

```
const nums = [10, 2, 30];
nums.sort((a, b) => a - b); // ascending numeric order
console.log(nums); // [2, 10, 30]
```

Note: sort() changes the original array. Without a comparator, values are sorted as strings.

10. Strings

A string is text surrounded by single quotes, double quotes or backticks.

```
const first = "Java";
const second = 'Script';
const language = `${first}${second}`;
console.log(language); // JavaScript
```

Property / method	Purpose
length	Number of characters.
toUpperCase()	Returns uppercase text.
toLowerCase()	Returns lowercase text.
trim()	Removes whitespace from both ends.
includes(text)	Checks whether text occurs inside.
startsWith(text)	Checks the beginning.
endsWith(text)	Checks the ending.
indexOf(text)	Finds the first position.
slice(start,end)	Extracts part of a string; end excluded.
replace(a,b)	Replaces the first matching occurrence (or matches with a global regex).
split(separator)	Splits a string into an array.

```
const message = "  Learn JavaScript  ";
console.log(message.trim());             // "Learn JavaScript"
console.log(message.toUpperCase());      // "  LEARN JAVASCRIPT  "
console.log("a,b,c".split(","));         // ["a", "b", "c"]
console.log("Hello".includes("ell"));    // true
```

Strings are immutable: string methods return a new string rather than changing the original string.

11. Objects

An object stores related information as key-value pairs. A key is also called a property name.

```
const student = {
  name: "Asha",
  age: 21,
  greet: function() {
    console.log("Hello!");
  }
};

console.log(student.name);     // dot notation
console.log(student["age"]);   // bracket notation
student.city = "Mumbai";       // add property
delete student.age;            // remove property
student.greet();
```

Object utilities

```
const person = { name: "Asha", age: 21 };

console.log(Object.keys(person));   // ["name", "age"]
console.log(Object.values(person)); // ["Asha", 21]
console.log(Object.entries(person)); // [["name","Asha"],["age",21]]
```

Destructuring

```
const user = { name: "Riya", age: 20 };
const { name, age } = user;

const colors = ["red", "blue"];
const [firstColor, secondColor] = colors;
```

Spread syntax

```
const oldArray = [1, 2];
const newArray = [...oldArray, 3]; // [1, 2, 3]

const oldUser = { name: "Asha" };
const newUser = { ...oldUser, city: "Pune" };
```

Spread makes a shallow copy. Nested objects are still shared unless they are copied separately.

12. Scope, Hoisting and Closures

Scope

Scope decides where a variable can be accessed.

```
const globalName = "Asha";

function show() {
  const localName = "Riya";
  console.log(globalName); // accessible
  console.log(localName);  // accessible here
}

show();
// console.log(localName); // ReferenceError
```

- Global scope: accessible in the relevant global environment.
- Function scope: available inside the function.
- Block scope: let and const are limited to the surrounding { } block.

Hoisting

Hoisting describes how JavaScript processes declarations before executing code. It does not mean every variable can safely be used before its declaration.

```
console.log(a); // undefined
var a = 10;

// console.log(b); // ReferenceError (temporal dead zone)
let b = 20;

sayHello(); // works
function sayHello() {
  console.log("Hello");
}
```

Closure

A closure happens when a function remembers and can access variables from the scope where it was created, even after that outer function has finished.

```
function makeCounter() {
  let count = 0;

  return function() {
    count++;
    return count;
  };
}

const counter = makeCounter();
console.log(counter()); // 1
console.log(counter()); // 2
```

13. this Keyword and Classes

The value of **this** depends on how a function is called. In a method call, this usually refers to the object before the dot. Arrow functions do not create their own this.

```
const user = {
  name: "Asha",
  showName() {
    console.log(this.name);
  }
};
user.showName(); // Asha
```

Classes

A class is a syntax for creating objects with shared structure and behaviour. JavaScript classes use prototypes underneath.

```
class Person {
  constructor(name) {
    this.name = name;
  }

  greet() {
    return `Hello, ${this.name}`;
  }
}

const person = new Person("Asha");
console.log(person.greet()); // Hello, Asha
```

Inheritance

```
class Animal {
  speak() {
    console.log("Animal sound");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Woof!");
  }
}

const dog = new Dog();
dog.speak();
dog.bark();
```

14. DOM (Document Object Model)

The DOM is the browser's object representation of an HTML document. JavaScript can use it to read and change page content, styles and elements.

```
// Find elements
const title = document.getElementById("title");
const item = document.querySelector(".item");

// Change content
title.textContent = "Welcome";

// Change style
title.style.color = "blue";

// Create and add an element
const para = document.createElement("p");
para.textContent = "A new paragraph";
document.body.appendChild(para);
```

Method / property	Purpose
getElementById()	Find an element by its id.
querySelector()	Find the first element matching a CSS selector.
querySelectorAll()	Find all matching elements.
textContent	Read or set text content.
innerHTML	Read or set HTML markup; be careful with untrusted input.
classList.add/remove/toggle()	Change CSS classes.
createElement()	Create a new element.
appendChild()	Add a child node.

Security tip: Prefer textContent for untrusted text. Assigning untrusted content to innerHTML can create cross-site scripting (XSS) vulnerabilities.

15. Events

An event is an action or occurrence in the browser, such as a click, key press, form submission or page load.

```
const button = document.querySelector("#save");

button.addEventListener("click", function() {
  console.log("Button clicked!");
});
```

Common events

- click — user clicks an element.
- input — a form control's value changes.
- change — a value is committed/changed.
- submit — a form is submitted.
- keydown — a key is pressed.
- mouseover — pointer moves onto an element.

Event object

```
button.addEventListener("click", function(event) {
  console.log(event.type);
  console.log(event.target);
});
```

Prevent default behaviour

```
const form = document.querySelector("form");

form.addEventListener("submit", function(event) {
  event.preventDefault(); // stops normal form navigation
  console.log("Handle form data here");
});
```

Event bubbling

In event bubbling, an event starts at the target element and then travels up through its ancestors. Event delegation uses this to handle events from many child elements with one listener on a parent.

16. Asynchronous JavaScript

JavaScript often needs to wait for work such as a timer or a network request. Asynchronous code lets the program continue instead of blocking while that work completes.

setTimeout and setInterval

```
setTimeout(() => {
  console.log("Runs once after about 1 second");
}, 1000);

const id = setInterval(() => {
  console.log("Runs repeatedly");
}, 1000);

// Stop it when no longer needed:
clearInterval(id);
```

Promise

A Promise represents the eventual success or failure of an asynchronous operation. Its states are pending, fulfilled or rejected.

```
const promise = new Promise((resolve, reject) => {
  const success = true;

  if (success) {
    resolve("Work completed");
  } else {
    reject(new Error("Something failed"));
  }
});

promise
  .then(result => console.log(result))
  .catch(error => console.error(error))
  .finally(() => console.log("Finished"));
```

async and await

An async function always returns a Promise. await pauses that async function until the awaited Promise settles; it does not block the entire JavaScript thread.

```
async function loadData() {
  try {
    const response = await fetch("https://example.com/data");
    if (!response.ok) {
      throw new Error("Request failed");
    }
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}

loadData();
```

Event loop: simple idea

JavaScript runs synchronous code on the call stack. Browser/Node features handle certain asynchronous work. When work is ready, callbacks are queued; the event loop schedules them when the stack is clear. Promise callbacks (microtasks) are generally processed before timer callbacks (tasks/macrotasks).

```
console.log("A");

setTimeout(() => console.log("B"), 0);

Promise.resolve().then(() => console.log("C"));

console.log("D");

// Output: A, D, C, B
```

17. JSON, Fetch and Web Storage

JSON

JSON (JavaScript Object Notation) is a text format commonly used to exchange data.

```
const user = { name: "Asha", age: 21 };

const jsonText = JSON.stringify(user); // object -> JSON string
const backToObject = JSON.parse(jsonText); // JSON string -> object
```

Fetch API

fetch() makes a network request and returns a Promise. Check response.ok because HTTP error statuses do not automatically reject the Promise.

```
fetch("https://example.com/users")
  .then(response => {
    if (!response.ok) throw new Error("HTTP error");
    return response.json();
  })
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

localStorage and sessionStorage

Storage	How long it lasts
localStorage	Persists across browser restarts until cleared.
sessionStorage	Usually lasts for the lifetime of the browser tab session.

```
localStorage.setItem("theme", "dark");
const theme = localStorage.getItem("theme");
localStorage.removeItem("theme");

// Store objects as JSON:
localStorage.setItem("user", JSON.stringify({ name: "Asha" }));
const user = JSON.parse(localStorage.getItem("user"));
```

Web Storage stores strings. Do not store passwords, secret tokens or sensitive information in it without understanding the security risks.

18. Error Handling and Regular Expressions

try, catch and finally

```
try {
  const data = JSON.parse("{invalid json}");
  console.log(data);
} catch (error) {
  console.log("Could not parse JSON");
} finally {
  console.log("This runs either way");
}
```

Use throw to create an error when an operation cannot continue as expected.

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }
  return a / b;
}
```

Regular expressions (RegExp)

A regular expression is a pattern used to search, match or replace text.

```
const text = "My number is 9876543210";
const digits = text.match(/\d+/g);
console.log(digits); // ["9876543210"]

console.log(/hello/i.test("Hello")); // true
```

Pattern	Meaning
\d	A digit from 0 to 9.
\w	A word character (letters, digits or underscore).
\s	Whitespace.
+	One or more of the preceding item.
*	Zero or more.
^	Beginning of input.
$	End of input.
[abc]	Any one of a, b or c.

19. Important ES6+ Features

Feature	Simple meaning
let and const	Block-scoped variable declarations.
Arrow functions	Short function syntax; lexical this.
Template literals	Backticks allow interpolation and multiline strings.
Destructuring	Extract values from arrays or properties from objects.
Spread syntax	Expand iterable values or object properties.
Rest parameters	Collect remaining arguments into an array.
Default parameters	Use a fallback when an argument is undefined.
Modules	Share code using export and import.
Optional chaining	Safely access a nested property.
Nullish coalescing	Fallback only for null or undefined.

Modules

```
// math.js
export function add(a, b) {
  return a + b;
}

// app.js
import { add } from "./math.js";
console.log(add(2, 3)); // 5
```

In a browser, use a module script: `<script type="module" src="app.js"></script>`.

20. Imperative vs Declarative Programming

Imperative	Declarative
Explains HOW to do a task.	Explains WHAT result is wanted.
Describes steps and control flow.	Describes the desired outcome.
Often uses loops and assignments.	Often uses methods or queries.

Imperative example:

```
const nums = [1, 2, 3, 4];
const evens = [];

for (let i = 0; i < nums.length; i++) {
  if (nums[i] % 2 === 0) {
    evens.push(nums[i]);
  }
}
```

Declarative example:

```
const nums = [1, 2, 3, 4];
const evens = nums.filter(num => num % 2 === 0);
```

Both produce [2, 4]. In the second example, filter describes the result wanted; JavaScript handles the iteration details.

21. Common Interview Questions

What is JavaScript?
A programming language used to add behaviour to webpages and to build applications in other environments.

Is JavaScript compiled or interpreted?
Modern engines commonly use multiple techniques, including interpretation and just-in-time compilation. It is not useful to describe all JavaScript execution as only one or the other.

Is JavaScript statically or dynamically typed?
Dynamically typed: a variable can refer to values of different types at different times.

What is NaN?
A special Number value meaning Not-a-Number. Use Number.isNaN() to test it reliably.

What is the difference between null and undefined?
undefined commonly means no value has been assigned; null is an intentional empty value.

What is the difference between == and ===?
== allows type coercion; === compares without coercion.

What is the difference between map and forEach?
map returns a new array; forEach runs a callback and returns undefined.

What is the difference between filter and find?
filter returns all matching elements in an array; find returns the first matching element or undefined.

What is the DOM?
The browser's object representation of an HTML document.

What is a callback?
A function passed to another function to be called later or during its work.

What is a Promise?
An object representing the eventual result of an asynchronous operation.

What does async do?
It makes a function return a Promise.

What does await do?
It waits for a Promise inside an async function and resumes that function when it settles.

What is a closure?
A function retaining access to variables from its outer lexical scope.

What is hoisting?
JavaScript's declaration-processing behaviour before code execution; var, let, const and function declarations behave

differently.

What is event bubbling?
An event travels from its target element up through ancestor elements.

What is the difference between localStorage and sessionStorage?
localStorage persists until cleared; sessionStorage is scoped to a tab's session.

What is the difference between slice and splice?
slice returns a portion without changing the original; splice changes the original array.

What is typeof null?
It is "object" due to a historical JavaScript quirk, although null is primitive.

What is the difference between function and arrow function?
Arrow functions have lexical this and no own arguments; regular functions have their own this depending on the call.

22. Practice Questions

Try solving these without looking at the answers first.

- Write a function that returns the square of a number.
- Count the even numbers in an array.
- Find the largest number in an array.
- Reverse a string.
- Check whether a number is a palindrome.
- Use map() to convert an array of prices from rupees to paise.
- Use filter() to keep numbers greater than 10.
- Use reduce() to calculate the sum of an array.
- Create an object for a student and print each key and value.
- Add a click event listener to a button that changes a heading.

Sample solutions

```
// 1. Square
const square = n => n * n;

// 2. Count even numbers
const countEvens = arr => arr.filter(n => n % 2 === 0).length;

// 3. Largest number
const largest = arr => Math.max(...arr);

// 4. Reverse a string
const reverse = str => str.split("").reverse().join("");

// 7. Values greater than 10
const greaterThanTen = arr => arr.filter(n => n > 10);

// 8. Sum
const sumArray = arr => arr.reduce((sum, n) => sum + n, 0);
```

23. Quick Revision Sheet

Concept	Remember
Variable	A named reference to a value.

Concept	Remember
Primitive	A simple value such as number, string or boolean.
Array	An ordered collection; indexes start at zero.
Object	A collection of key-value properties.
Function	Reusable code that can be called.
Callback	A function passed to another function.
Closure	A function remembers its outer lexical variables.
DOM	The browser's object representation of a page.
Event	An action such as click, input or submit.
Promise	Represents a future asynchronous result.
async/await	Syntax for working with Promises.
map	Transform each array element into a new array.
filter	Keep array elements that pass a test.
reduce	Combine array values into one result.
Declarative	Describe what result you want.
Imperative	Describe the steps to get the result.

Best way to learn: Do not only read examples. Type them, change their values, predict the output, and build small projects such as a calculator, to-do list, quiz app and form validator.